

# Unit V- Memory Management

## Topics to cover

### Main Memory

Background

Swapping

Contiguous Memory Allocation

Non Contiguous Memory Allocation

Segmentation Paging Structure of the Page Table

### Virtual Memory

Background

Demand Paging

Page Replacement

Allocation of Frames

Page replacement algorithms: -FIFO, LRU, OPT Thrashing

Memory management is the functionality of an operating system which handles or manages primary memory and moves processes back and forth between main memory and disk during execution. Memory management keeps track of each and every memory location, regardless of either it is allocated to some process or it is free.

It checks how much memory is to be allocated to processes.

It decides which process will get memory at what time.

It tracks whenever some memory gets freed or unallocated and correspondingly it updates the status.

## Main memory

It is also known as RAM (Random Access Memory), is the primary, volatile storage where the CPU directly accesses data and instructions for running programs.

The main memory is the fundamental storage unit in a computer system. It is associatively large and quick memory and saves programs and information during computer operations. The technology that makes the main memory work is based on semiconductor integrated circuits.

RAM is the main memory. Integrated circuit Random Access Memory (RAM) chips are applicable in two possible operating modes are as follows

- **Static** It consists of internal flip-flops, which store the binary information. The stored data remains solid considering power is provided to the unit. The static RAM is simple to use and has smaller read and write cycles.
- **Dynamic** It saves the binary data in the structure of electric charges that are used to capacitors. The capacitors are made available inside the chip by Metal

Oxide Semiconductor (MOS) transistors. The stored value on the capacitors contributes to discharge with time and thus, the capacitors should be regularly recharged through stimulating the dynamic memory.

## Random Access Memory

The term **Random Access Memory** or RAM is typically used to refer to memory that is easily read from and written to by the microprocessor. For a memory to be called random access, it should be possible to access any address at any time. This differentiates RAM from storage devices such as tapes or hard drives where the data is accessed sequentially.

RAM is the main memory of a computer. Its objective is to store data and applications that are currently in use.

The **operating system** controls the usage of this memory. It gives instructions like when the items are to be loaded into RAM, where they are to be located in RAM, and when they need to be removed from RAM.

## Read-Only Memory

In each computer system, there should be a segment of memory that is fixed and unaffected by power failure. This type of memory is known as **Read-Only Memory** or ROM.

## SRAM

RAMs that are made up of circuits and can preserve the information as long as power is supplied are referred to as **Static Random Access Memories (SRAM)**.

Flip-flops form the basic memory elements in an SRAM device. An SRAM consists of an array of flip-flops, one for each bit.

SRAM consists of an array of flip-flops; a large number of flip-flops are needed to provide higher capacity memory. Because of this, simpler flip-flop circuits, BJT, and MOS transistors are used for SRAM.

## DRAM

SRAMs are faster but their cost is high because their cells require many transistors. RAMs can be obtained at a lower cost if simpler cells are used. A MOS storage cell based on capacitors can be used to replace the SRAM cells. Such a storage cell cannot preserve the charge (that is, data) indefinitely and must be recharged periodically.

Therefore, these cells are called dynamic storage cells. RAMs using these cells are referred to as Dynamic RAMs or simply DRAMs.

## Process Address Space

The process address space **is the set of logical addresses that a process references in its code**. For example, when 32-bit addressing is in use,

addresses can range from 0 to 0x7fffffff; that is,  $2^{31}$  possible numbers, for a total theoretical size of 2 gigabytes.

The operating system takes care of mapping the logical addresses to physical addresses at the time of memory allocation to the program. There are three types of addresses used in a program before and after memory is allocated –

| S.N | Memory Addresses & Description                                                                                                                                                |
|-----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1   | <b>Symbolic addresses</b><br>The addresses used in a source code. The variable names, constants, and instruction labels are the basic elements of the symbolic address space. |
| 2   | <b>Relative addresses</b><br>At the time of compilation, a compiler converts symbolic addresses into relative addresses.                                                      |
| 3   | <b>Physical addresses</b><br>The loader generates these addresses at the time when a program is loaded into main memory.                                                      |

Virtual and physical addresses are the same in compile-time and load-time address-binding schemes. Virtual and physical addresses differ in execution-time address-binding scheme.

The set of all logical addresses generated by a program is referred to as a **logical address space**. The set of all physical addresses corresponding to these logical addresses is referred to as a **physical address space**.

The runtime mapping from virtual to physical address is done by the memory management unit (MMU) which is a hardware device. MMU uses following mechanism to convert virtual address to physical address.

- The value in the base register is added to every address generated by a user process, which is treated as offset at the time it is sent to memory. For example, if the base register value is 10000, then an attempt by the user to use address location 100 will be dynamically reallocated to location 10100.
- The user program deals with virtual addresses; it never sees the real physical addresses.

## Static vs Dynamic Loading

The choice between Static or Dynamic Loading is to be made at the time of computer program being developed. If you have to load your program

statically, then at the time of compilation, the complete programs will be compiled and linked without leaving any external program or module dependency. The linker combines the object program with other necessary object modules into an absolute program, which also includes logical addresses.

If you are writing a Dynamically loaded program, then your compiler will compile the program and for all the modules which you want to include dynamically, only references will be provided and rest of the work will be done at the time of execution.

At the time of loading, with **static loading**, the absolute program (and data) is loaded into memory in order for execution to start.

If you are using **dynamic loading**, dynamic routines of the library are stored on a disk in relocatable form and are loaded into memory only when they are needed by the program.

## Static vs Dynamic Linking

As explained above, when static linking is used, the linker combines all other modules needed by a program into a single executable program to avoid any runtime dependency.

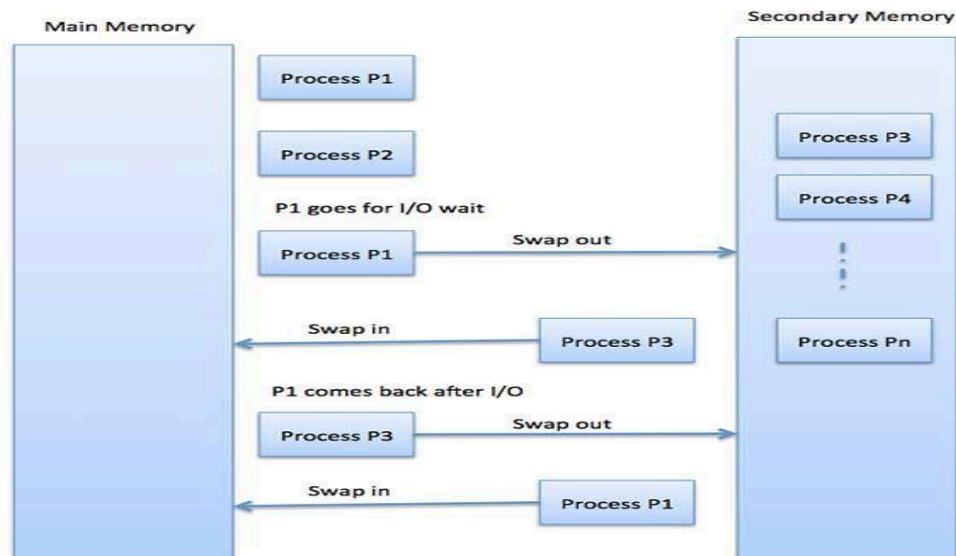
When dynamic linking is used, it is not required to link the actual module or library with the program, rather a reference to the dynamic module is provided at the time of compilation and linking. **Dynamic Link Libraries (DLL) in Windows and Shared Objects in Unix** are good examples of dynamic libraries.

## Swapping

Swapping is a mechanism in which a process can be swapped temporarily out of main memory (or move) to secondary storage (disk) and make that memory available to other processes.

At some later time, the system swaps back the process from the secondary storage to main memory.

Though performance is usually affected by swapping process but it helps in running multiple and big processes in parallel and that's the reason **Swapping is also known as a technique for memory compaction.**



The total time taken by swapping process includes the time it takes to move the entire process to a secondary disk and then to copy the process back to memory, as well as the time the process takes to regain main memory.

Let us assume that the user process is of size 2048KB and on a standard hard disk where swapping will take place has a data transfer rate around 1 MB per second. The actual transfer of the 1000K process to or from memory will take

$2048\text{KB} / 1024\text{KB per second}$   
 $= 2 \text{ seconds}$   
 $= 2000 \text{ milliseconds}$

Now considering in and out time, it will take complete 4000 milliseconds plus other overhead where the process competes to regain main memory.

## Memory Allocation

Main memory usually has two partitions –

- **Low Memory** – Operating system resides in this memory.
- **High Memory** – User processes are held in high memory.

Operating system uses the following memory allocation mechanism.

S.N

Memory Allocation & Description

.

1

Single-partition allocation

In this type of allocation, relocation-register scheme is used to protect user processes from each other, and from changing operating-system code and data. Relocation register contains value of smallest physical address whereas limit register contains range of logical addresses. Each logical address must be less than the limit register.

#### **Multiple-partition allocation**

2

In this type of allocation, main memory is divided into a number of fixed-sized partitions where each partition should contain only one process. When a partition is free, a process is selected from the input queue and is loaded into the free partition. When the process terminates, the partition becomes available for another process.

## Fragmentation

As processes are loaded and removed from memory, the free memory space is broken into little pieces. It happens after sometimes that processes cannot be allocated to memory blocks considering their small size and memory blocks remains unused. This problem is known as Fragmentation.

Fragmentation is of two types –

S.N

### **Fragmentation & Description**

.

#### **External fragmentation**

1 Total memory space is enough to satisfy a request or to reside a process in it, but it is not contiguous, so it cannot be used.

#### **Internal fragmentation**

2 Memory block assigned to process is bigger. Some portion of memory is left unused, as it cannot be used by another process.

The following diagram shows how fragmentation can cause waste of memory and a compaction technique can be used to create more free memory out of fragmented memory –

Fragmented memory before compaction



Memory after compaction



External fragmentation can be reduced by compaction or shuffle memory contents to place all free memory together in one large block. To make compaction feasible, relocation should be dynamic.

The internal fragmentation can be reduced by effectively assigning the smallest partition but large enough for the process.

## Contiguous memory allocation

Contiguous memory allocation **allocates a single, continuous block of memory to a process**, while **non-contiguous memory allocation allows a process to have its different parts residing in various, non-adjacent memory locations**.

Contiguous allocation offers **faster execution and easier management**, but can lead to external fragmentation, while non-contiguous allocation avoids fragmentation but may introduce overhead due to pointer usage.

Contiguous Memory Allocation:

- **Single Block:** Assigns a single, continuous segment of memory to a process.
- **Advantage:** Simpler to manage, potentially faster memory access, and good for sequential processing.
- **Disadvantage:** Can lead to external fragmentation (wasted space) if a process needs to be removed, leaving a fragmented free space.

Non-Contiguous Memory Allocation:

- **Multiple Blocks:** Allows a process to be stored in multiple, non-adjacent memory locations.
- **Advantage:** Avoids external fragmentation, as a process can be placed in available memory blocks even if they are not contiguous.
- **Disadvantage:** May introduce overhead due to the need for pointers to track the location of different parts of the process, potentially slowing down access times.
- Difference between Contiguous and Noncontiguous Memory Allocation

In contiguous memory allocation, a single part of the memory section is allowed to accomplish the further process. While on other ...

## Contiguous Memory Allocation

Contiguous memory allocation is a technique where the operating system allocates a contiguous block of memory to a process. This memory is allocated in a single, continuous chunk, making it easy for the operating system to manage and for the process to access the memory. Contiguous memory allocation is suitable for systems with limited memory sizes and where fast access to memory is important.

### Contiguous memory allocation can be done in two ways

- **Fixed Partitioning** In fixed partitioning, the memory is divided into fixed-size partitions, and each partition is assigned to a process. This technique is easy to implement but can result in wasted memory if a process does not fit perfectly into a partition.
- **Dynamic Partitioning:** In dynamic partitioning, the memory is divided into variable size partitions, and each partition is assigned to a process. This technique is more efficient as it allows the allocation of only the required memory to the process, but it requires more overhead to keep track of the available memory.

### Advantages of Contiguous Memory Allocation

- **Simplicity** Contiguous memory allocation is a relatively simple and straightforward technique for memory management. It requires less overhead and is easy to implement.
- **Efficiency** Contiguous memory allocation is an efficient technique for memory management. Once a process is allocated contiguous memory, it can access the entire memory block without any interruption.
- **Low fragmentation** Since the memory is allocated in contiguous blocks, there is a lower risk of memory fragmentation. This can result in better memory utilization, as there is less memory wastage.

### Disadvantages of Contiguous Memory Allocation

- **Limited flexibility** Contiguous memory allocation is not very flexible as it requires memory to be allocated in a contiguous block. This can limit the amount of memory that can be allocated to a process.
- **Memory wastage** If a process requires a memory size that is smaller than the contiguous block allocated to it, there may be unused memory, resulting in memory wastage.
- **Difficulty in managing larger memory sizes** As the size of memory increases, managing contiguous memory allocation becomes more difficult. This is because



finding a contiguous block of memory that is large enough to allocate to a process becomes challenging.

- **External Fragmentation** Over time, external fragmentation may occur as a result of memory allocation and deallocation, which may result in non-contiguous blocks of free memory scattered throughout the system.

Overall, contiguous memory allocation is a useful technique for memory management in certain circumstances, but it may not be the best solution in all situations, particularly when working with larger amounts of memory or if flexibility is a priority.

## Non-contiguous Memory Allocation

Non-contiguous memory allocation, on the other hand, is a technique where the operating system allocates memory to a process in non-contiguous blocks. The blocks of memory allocated to the process need not be contiguous, and the operating system keeps track of the various blocks allocated to the process. Non-contiguous memory allocation is suitable for larger memory sizes and where efficient use of memory is important.

### Non-contiguous memory allocation can be done in two ways

- **Paging** in paging, the memory is divided into fixed-size pages, and each page is assigned to a process. This technique is more efficient as it allows the allocation of only the required memory to the process.
- **Segmentation** in segmentation, the memory is divided into variable-sized segments, and each segment is assigned to a process. This technique is more flexible than paging but requires more overhead to keep track of the allocated segments.

Non-contiguous memory allocation is a memory management technique that divides memory into non-contiguous blocks, allowing processes to be allocated memory that is not necessarily contiguous. Here are some of the advantages and disadvantages of noncontiguous memory allocation

### Advantages of Non-Contiguous Memory Allocation

- **Reduced External Fragmentation** ? One of the main advantages of non-contiguous memory allocation is that it can reduce external fragmentation, as memory can be allocated in small, non-contiguous blocks.
- **Increased Memory Utilization** ? Non-contiguous memory allocation allows for more efficient use of memory, as small gaps in memory can be filled with processes that need less memory.
- **Flexibility** ? This technique allows for more flexibility in allocating and deallocating memory, as processes can be allocated memory that is not necessarily contiguous.

- **Memory Sharing** ? Non-contiguous memory allocation makes it easier to share memory between multiple processes, as memory can be allocated in non-contiguous blocks that can be shared between multiple processes.

## Disadvantages of Non-Contiguous Memory Allocation

- **Internal Fragmentation** ? One of the main disadvantages of non-contiguous memory allocation is that it can lead to internal fragmentation, as memory can be allocated in small, non-contiguous blocks that are not fully utilized.
- **Increased Overhead** ? This technique requires more overhead than contiguous memory allocation, as the operating system needs to maintain data structures to track memory allocation.
- **Slower Access** ? Access to memory can be slower than contiguous memory allocation, as memory can be allocated in non-contiguous blocks that may require additional steps to access.

In summary, non-contiguous memory allocation has advantages such as reduced external fragmentation, increased memory utilization, flexibility, and memory sharing. However, it also has disadvantages such as internal fragmentation, increased overhead, and slower access to memory. Operating systems must carefully consider the trade-offs between these advantages and disadvantages when selecting memory management techniques.

## Difference between contiguous and non contiguous memory allocation in operating system

| Aspect       | Contiguous Memory Allocation                                                  | Non-Contiguous Memory Allocation                                                                                    |
|--------------|-------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| Method       | Allocates memory in a contiguous block to a process                           | Allocates memory to a process in non-contiguous blocks                                                              |
| Block Size   | Memory allocated in a single, continuous chunk                                | Memory allocated in noncontiguous blocks of varying sizes                                                           |
| Management   | Easy to manage by the operating system                                        | Requires additional overhead and can be more complicated to manage                                                  |
| Memory Usage | May result in memory wastage and external fragmentation                       | Efficient use of memory and reduces fragmentation within memory blocks                                              |
| Suitable For | Systems with limited amounts of memory and fast access to memory is important | Larger memory sizes and systems that require more efficient use of memory                                           |
| Advantages   | Simple and efficient technique for memory management                          | More flexible and efficient technique for larger memory sizes and systems that require more efficient use of memory |

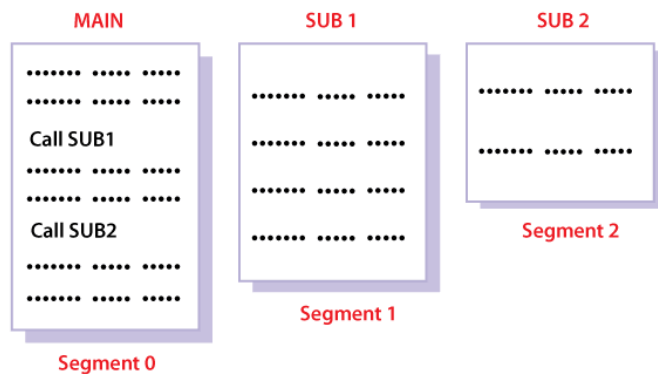
Disadvantages

Can be inflexible and result in memory wastage and fragmentation

Requires additional overhead and can be more complicated to manage

**Segmentation in OS** is a memory management technique in which the process is divided into logically related partitions or segments and then allocated to the memory. Each segment represents a different portion of a program or process, such as code, data, or stack.

It is better to have segmentation which divides the process into the segments. Each segment contains the same type of functions such as the main function can be included in one segment and the library functions can be included in the other segment.



## Paging

A computer can address more memory than the amount physically installed on the system. This extra memory is actually called virtual memory and it is a section of a hard disk that's set up to emulate the computer's RAM. Paging technique plays an important role in implementing virtual memory.

Paging is a memory management technique in which process address space is broken into blocks of the same size called **pages** (size is power of 2, between 512 bytes and 8192 bytes). The size of the process is measured in the number of pages.

Similarly, main memory is divided into small fixed-sized blocks of (physical) memory called **frames** and the size of a frame is kept the same as that of a page to have optimum utilization of the main memory and to avoid external fragmentation.

Paging is a memory management scheme that eliminates the need for a contiguous allocation of physical memory.

The process of retrieving processes in the form of pages from the secondary storage into the main memory is known as paging.

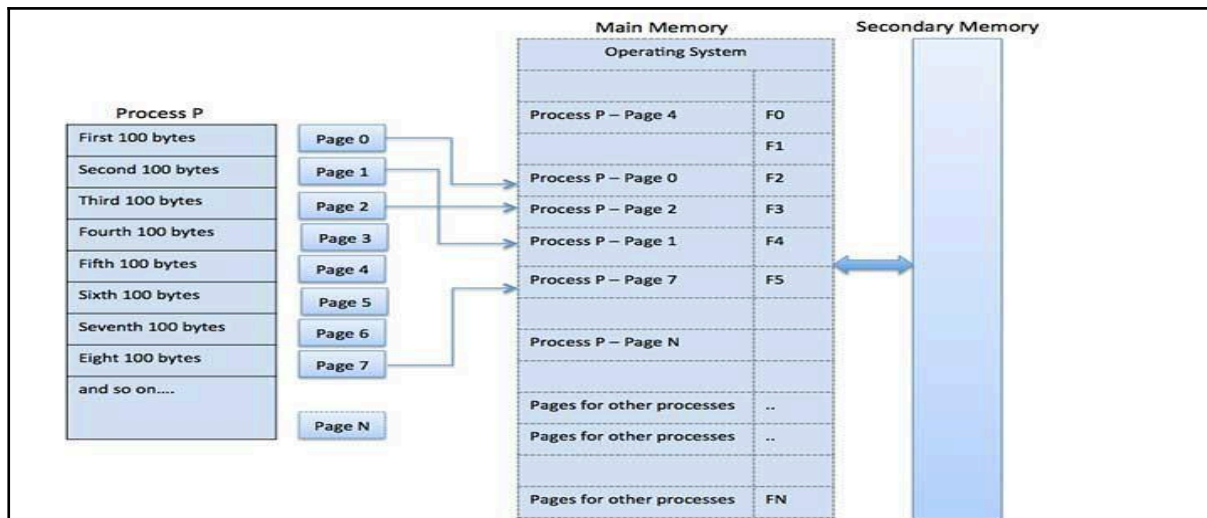
The basic purpose of paging is to separate each procedure into pages.

The mapping between logical pages and physical page frames is maintained by the page table, which is used by the memory management unit to translate logical addresses into physical addresses. The page table maps each logical page number to a physical page frame number.

By using a Page Table, the operating system keeps track of the mapping between logical addresses (used by programs) and physical addresses (actual locations in memory).

### Purpose of Paging in Computer System

- **Memory isn't always available in a single block:** Programs often need more memory than what is available in a single continuous block. Paging breaks memory into smaller, fixed-size pieces, making it easier to allocate scattered free spaces.
- **Processes size can increase or decrease:** programs don't need to occupy continuous memory, so they can grow dynamically without the need to be moved.



## Address Translation

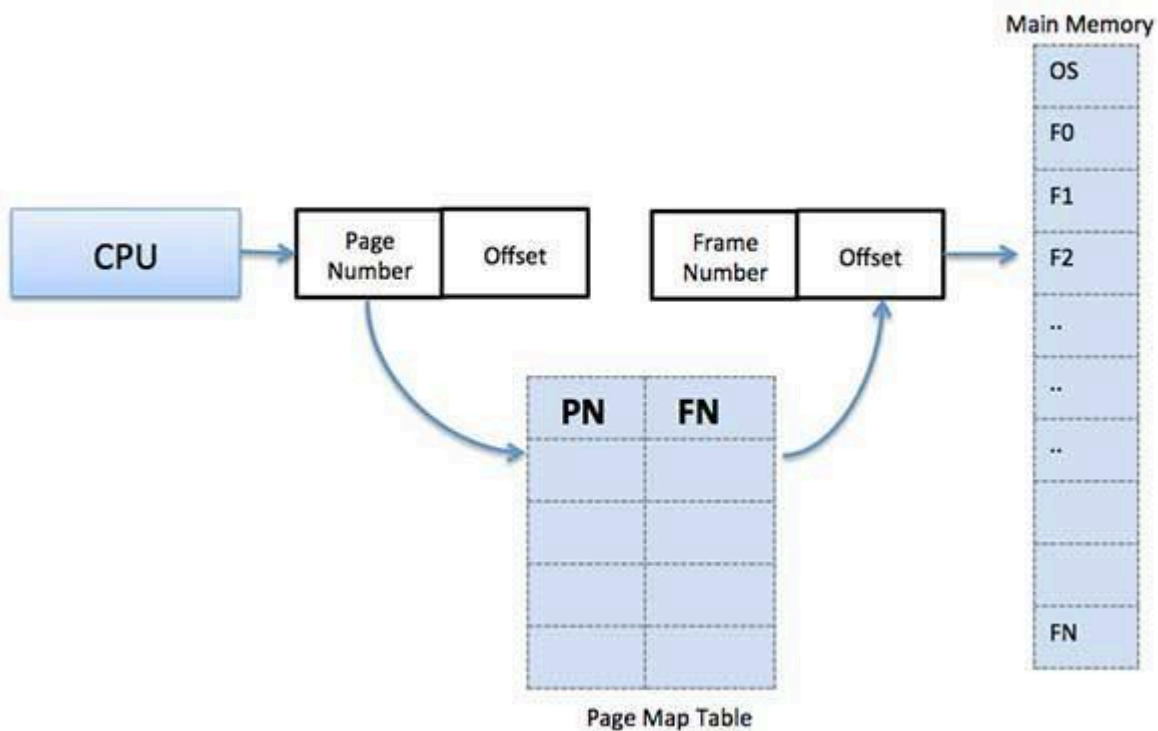
Page address is called **logical address** and represented by **page number** and the **offset**.

Logical Address = Page number + page offset

Frame address is called **physical address** and represented by a **frame number** and the **offset**.

Physical Address = Frame number + page offset

A data structure called **page map table** is used to keep track of the relation between a page of a process to a frame in physical memory.



When the system allocates a frame to any page, it translates this logical address into a physical address and create entry into the page table to be used throughout execution of the program.

When a process is to be executed, its corresponding pages are loaded into any available memory frames.

Suppose you have a program of 8Kb but your memory can accommodate only 5Kb at a given point in time, then the paging concept will come into picture. When a computer runs out of RAM, the operating system (OS) will move idle or unwanted pages of memory to secondary memory to free up RAM for other processes and brings them back when needed by the program.

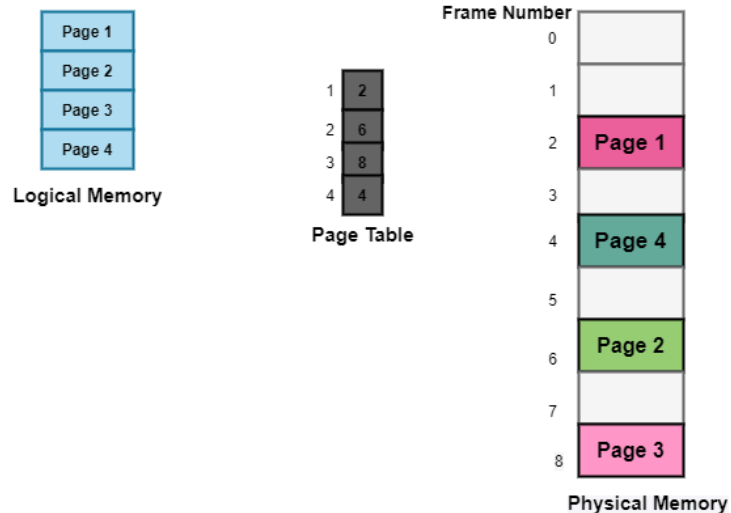
This process continues during the whole execution of the program where the OS keeps removing idle pages from the main memory and write them onto the secondary memory and bring them back when required by the program.

The data structure that is used by the virtual memory system in the operating system of a computer in order to store the mapping between physical and logical addresses is commonly known as **Page Table**.

The logical address that is generated by the CPU is translated into the physical address with the help of the page table.

- Thus page table mainly provides the corresponding frame number (base address of the frame) where that page is stored in the main memory.

The above diagram shows the paging model of Physical and logical memory.



## Characteristics of the Page Table

Some of the characteristics of the Page Table are as follows:

- It is stored in the main memory.
- Generally; the Number of entries in the page table = the Number of Pages in which the process is divided.
- **PTBR** means page table base register and it is basically used to hold the base address for the page table of the current process.
- Each process has its own independent page table.

## Advantages and Disadvantages of Paging

Here is a list of advantages and disadvantages of paging –

- Paging reduces external fragmentation, but still suffer from internal fragmentation.
- Paging is simple to implement and assumed as an efficient memory management technique.
- Due to equal size of the pages and frames, swapping becomes very easy.
- Page table requires extra memory space, so may not be good for a system having small RAM.

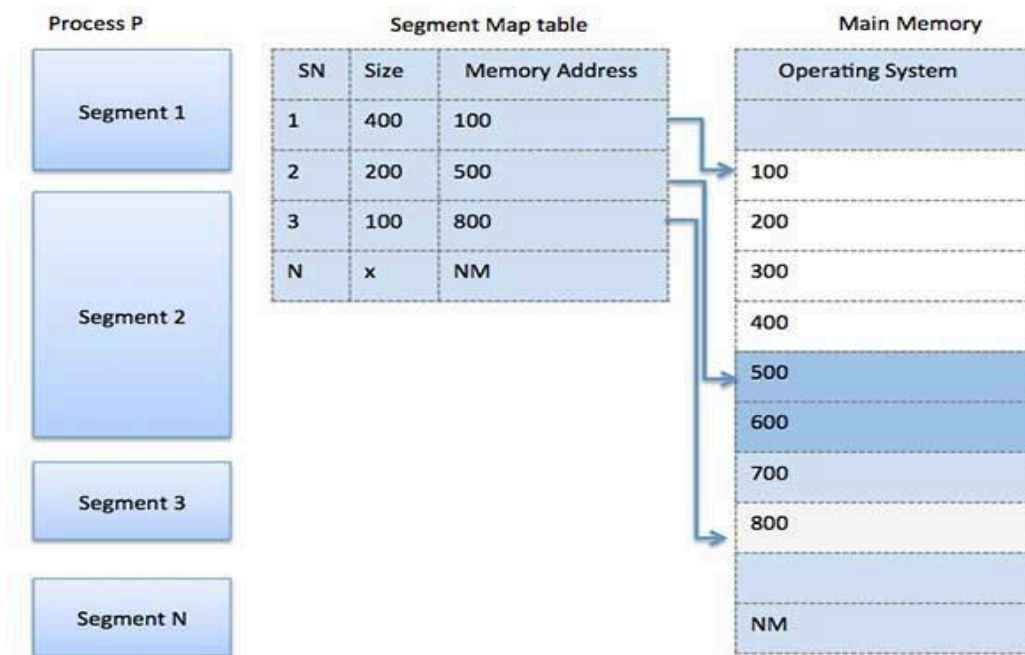
## Segmentation

Segmentation is a memory management technique in which each job is divided into several segments of different sizes, one for each module that contains pieces that perform related functions. Each segment is actually a different logical address space of the program.

When a process is to be executed, its corresponding segmentation are loaded into non-contiguous memory though every segment is loaded into a contiguous block of available memory.

Segmentation memory management works very similar to paging but here segments are of variable-length whereas in paging pages are of fixed size.

A program segment contains the program's main function, utility functions, data structures, and so on. The operating system maintains a **segment map table** for every process and a list of free memory blocks along with segment numbers, their size and corresponding memory locations in main memory. For each segment, the table stores the starting address of the segment and the length of the segment. A reference to a memory location includes a value that identifies a segment and an offset.



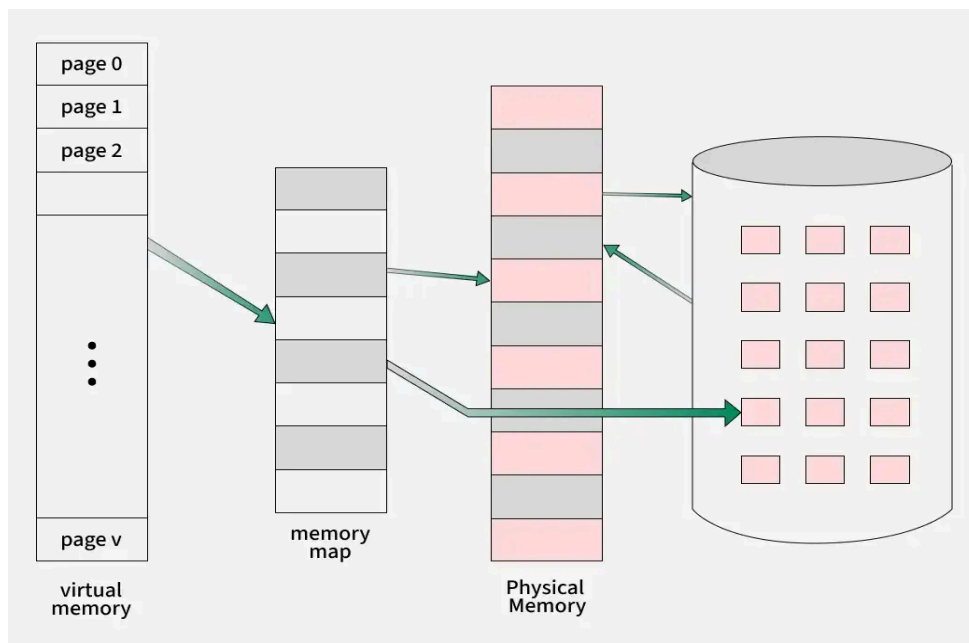
## Virtual Memory

### Virtual Memory in Operating System

- Virtual memory is a memory management technique used by operating systems to give the appearance of a large, continuous block of memory to applications, even if the physical memory (RAM) is limited.
- It allows larger applications to run on systems with less RAM.



- The main objective of virtual memory is to support multiprogramming,
- The main advantage that virtual memory provides is, a running process does not need to be entirely in memory.
- Programs can be larger than the available physical memory. Virtual Memory provides an abstraction of main memory, eliminating concerns about storage limitations.
- A memory hierarchy, consisting of a computer system's memory and a disk, enables a process to operate with only some portions of its address space in RAM to allow more processes to be in memory.



A virtual memory is what its name indicates- it is an illusion of a memory that is larger than the real memory.

The basis of virtual memory is the non-contiguous memory allocation model.

The virtual memory manager removes some components from memory to make room for other components.

The size of virtual storage is limited by the addressing scheme of the computer system and the amount of secondary memory available not by the actual number of main storage locations.

Virtual Memory is a technique that is implemented using both hardware and software. It maps memory addresses used by a program, called virtual addresses, into physical addresses in computer memory.

- All memory references within a process are logical addresses that are dynamically translated into physical addresses at run time.
- This means that a process can be swapped in and out of the main memory such that it occupies different places in the main memory at different times during the course of execution.
- A process may be broken into a number of pieces and these pieces need not be continuously located in the main memory during execution. The combination of dynamic run-time address translation and the use of a page or segment table permits this.

If these characteristics are present then, it is not necessary that all the pages or segments are present in the main memory during execution.

This means that the required pages need to be loaded into memory whenever required.

Virtual memory is implemented using Demand Paging or Demand Segmentation.

## Paging

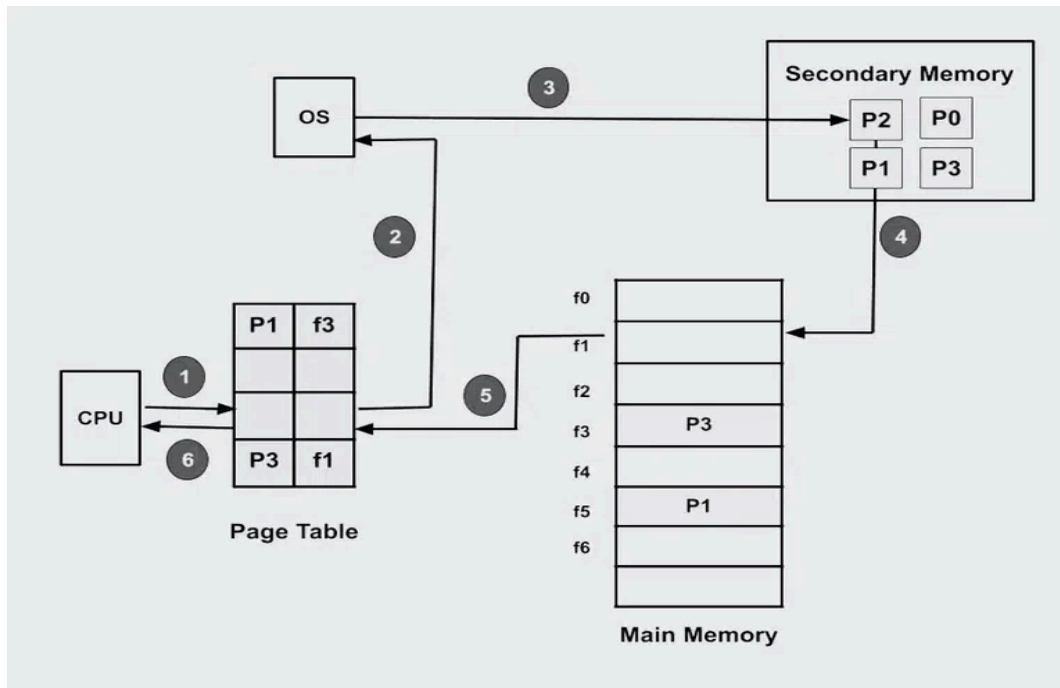
Paging divides memory into small fixed-size blocks called pages.

When the computer runs out of RAM, pages that aren't currently in use are moved to the hard drive, into an area called a swap file.

The swap file acts as an extension of RAM.

When a page is needed again, it is swapped back into RAM, a process known as page swapping.

This ensures that the operating system (OS) and applications have enough memory to run.



## Demand Paging:

The process of loading the page into memory on demand (whenever a page fault occurs) is known as **demand paging**

The process includes the following steps

1. If the CPU tries to refer to a page that is currently not available in the main memory, it generates an interrupt indicating a memory access fault.
2. The OS puts the interrupted process in a blocking state. For the execution to proceed the OS must bring the required page into the memory.
3. The OS will search for the required page in the logical address space.
4. The required page will be brought from logical address space to physical address space. **The page replacement algorithms** are used for the decision-making of replacing the page in physical address space.
5. The page table will be updated accordingly.
6. The signal will be sent to the CPU to continue the program execution and it will place the process back into the ready state.

## Page vs Frame

A page is a fixed size block of data in virtual memory and a frame is a fixed size block of physical memory in RAM where these pages are loaded.

Think of a page as a piece of a puzzle (virtual memory) and a frame as the spot where it fits on the board (physical memory).

When a program runs its pages are mapped to available frames so the program can run even if the program size is larger than physical memory.

### Virtual Memory vs Physical Memory

| Feature     | Virtual Memory                                                         | Physical Memory (RAM)                                                                       |
|-------------|------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| Definition  | An abstraction that extends the available memory by using disk storage | The actual hardware (RAM) that stores data and instructions currently being used by the CPU |
| Location    | On the hard drive or SSD                                               | On the computer's motherboard                                                               |
| Speed       | Slower (due to disk I/O operations)                                    | Faster (accessed directly by the CPU)                                                       |
| Capacity    | Larger, limited by disk space                                          | Smaller, limited by the amount of RAM installed                                             |
| Cost        | Lower (cost of additional disk storage)                                | Higher (cost of RAM modules)                                                                |
| Data Access | Indirect (via paging and swapping)                                     | Direct (CPU can access data directly)                                                       |
| Volatility  | Non-volatile (data persists on disk)                                   | Volatile (data is lost when power is off)                                                   |

## Demand Paging in Operating System



Demand paging is a memory management scheme used in operating systems to improve memory usage and system performance.

### Let's understand demand paging with real life example

Imagine you are reading a very thick book, but you don't want to carry the entire book around because it's too heavy. Instead, you decide to only bring the pages you need as you read through the book. When you finish with one page, you can put it away and grab the next page you need.

In a computer system, the book represents the entire program, and the pages are parts of the program called "pages" of memory.

Demand paging works similarly: instead of loading the whole program into the computer's memory at once (which can be very large and take up a lot of space), the operating system only loads the necessary parts (pages) of the program when they are needed.

This concept says that we should not load any pages into the main memory until we need them, or keep all pages in secondary memory until we need them.

Demand paging is a technique used in virtual memory systems where pages enter main memory only when requested or needed by the CPU.

In demand paging, the operating system loads only the necessary pages of a program into memory at runtime, instead of loading the entire program into memory at the start.

A page fault occurred when the program needed to access a page that is not currently in memory.

The operating system then loads the required pages from the disk into memory and updates the page tables accordingly.

This process is transparent to the running program and it continues to run as if the page had always been in memory.

## Page Fault

The term "page miss" or "Page fault" refers to a situation where a referenced page is not found in the main memory.

When a program tries to access a page, or fixed-size block of memory, that isn't currently loaded in physical memory (RAM), an exception known as a page fault happens. Before enabling the program to access a page that is required, the operating system must bring it into memory from secondary storage (such a hard drive) in order to handle a page fault.

In modern operating systems, page faults are a common component of virtual memory management. By enabling programs to operate with more data than can fit in physical memory at once, they enable the efficient use of physical memory. The operating system is responsible for coordinating the transfer of data between physical memory and secondary storage as needed.

## Thrashing

Thrashing is the term used to describe a state in which excessive paging activity takes place in computer systems, especially in operating systems that use virtual memory, severely impairing system performance. Thrashing occurs when a system's high memory demand and low physical memory capacity cause it to spend a large amount of time rotating pages between main memory (RAM) and secondary storage, which is typically a hard disc.

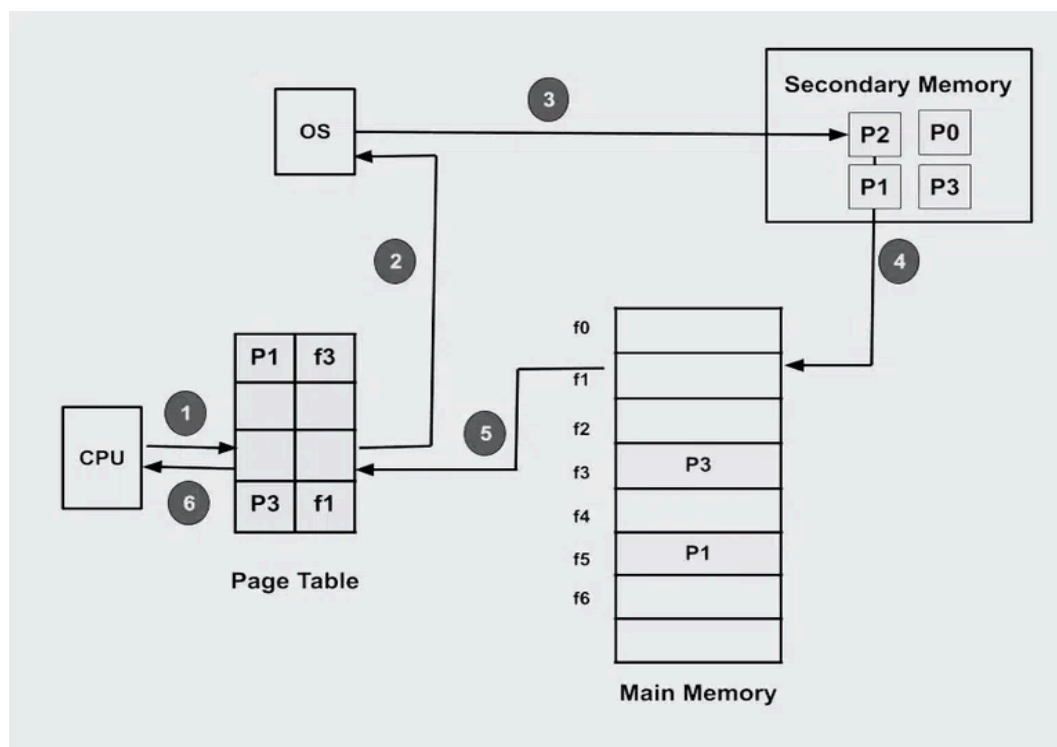
It is caused due to insufficient physical memory, overloading and poor memory management.

The operating system may use a variety of techniques to lessen thrashing, including lowering the number of running processes, adjusting paging parameters, and improving memory allocation algorithms.

Increasing the system's physical memory (RAM) capacity can also lessen thrashing by lowering the frequency of page swaps between RAM and the disc.

## Working of Demand Paging

Let us understand this with the help of an example. Suppose we want to run a process P which have four pages P0, P1, P2, and P3. Currently, in the page table, we have pages P1 and P3.



The Operating System's demand paging mechanism follows a few steps in its operation.

- **Program Execution:** Upon launching a program, the operating system allocates a certain amount of memory to the program and establishes a process for it.
- **Creating Page Tables:** To keep track of which program pages are currently in memory and which are on disk, the operating system makes [page tables](#) for each process.
- **Handling Page Fault:** When a program tries to access a page that isn't in memory at the moment, a page fault happens. In order to determine whether the necessary page is on disk, the operating system pauses the application and consults the page tables.
- **Page Fetch:** The operating system loads the necessary page into memory by retrieving it from the disk if it is there.
- The page's new location in memory is then reflected in the page table.
- **Resuming The Program:** The operating system picks up where it left off when the necessary pages are loaded into memory.
- **Page Replacement:** If there is not enough free memory to hold all the pages a program needs, the operating system may need to replace one or more pages currently in memory with pages currently in memory. on the disk. The page replacement algorithm used by the operating system determines which pages are selected for replacement.
- **Page Cleanup:** When a process terminates, the operating system frees the memory allocated to the process and cleans up the corresponding entries in the page tables.

## Steps to Create a Page Map Table (Page Table)

### Step 1: Divide the Logical Address Space

- Split the **process (logical) address space** into **fixed-size pages**.
  - Page size is usually a power of 2 (e.g., 4 KB).
  - Number pages as **Page 0, Page 1, Page 2, ...**
- 

### Step 2: Divide the Physical Memory

- Divide **main memory** into **frames** of the **same size** as pages.
  - Frames are numbered **Frame 0, Frame 1, Frame 2, ...**
- 

### Step 3: Allocate Frames to Pages

- Allocate **free frames** in physical memory to the pages of the process.
- Allocation may be **non-contiguous**.

---

## Step 4: Create Page Table Entries

For each page, create an entry in the **Page Map Table** with:

- **Page Number**
  - **Frame Number** (where the page is stored)
  - Optional control bits:
    - **Valid / Invalid bit**
    - **Protection bits (R/W/X)**
    - **Dirty bit**
    - **Reference bit**
- 

## Step 5: Fill the Page Table

- Map every **logical page number** → **physical frame number**.
  - If a page is not in memory, mark it **invalid**.
- 

## Step 6: Store Page Table in Memory

- The page table is stored in **main memory**.
  - A **Page Table Base Register (PTBR)** holds the starting address of the page table.
- 

## Step 7: Address Translation Using Page Table

- Logical address is divided into:
    - **Page Number (p)**
    - **Offset (d)**
  - Use **p** to index the page table and get the **frame number (f)**.
  - Physical address = **(frame number × frame size) + offset**
- 

## Example Page Map Table

| Page Number | Frame Number | Valid Bit |
|-------------|--------------|-----------|
| 0           | 5            | 1         |
| 1           | 2            | 1         |
| 2           | 9            | 1         |
| 3           | —            | 0         |



## Overview of Virtual Memory

Before delving into web page replacement algorithms, it is important to understand the idea of digital reminiscence. Virtual memory is a reminiscence manipulation method that offers software the illusion of having a large contiguous block of memory, regardless of the fact that the bodily memory is fragmented or loads a whole lot less than the favored reminiscence. The OS manages virtual memory with the resource of dividing it into pages, commonly four KB in duration, and mapping them to bodily reminiscence frames.

When software accesses a memory location, the memory manipulation unit (MMU) translates the digital address to a bodily deal. Suppose the internet web page is not in bodily memory. In that case, an internet web page fault takes vicinity, and the OS should supply the internet net page from a secondary garage (like tough stress) into physical reminiscence.

## Need for Page Replacement

When a web page fault takes place, and the bodily memory is complete, the OS wants to decide which net web page to evict to make room for the cutting-edge web page. This selection is essential because the wrong choice can lead to inefficient memory use and slow device overall performance. Page substitute algorithms aim to make top-rated decisions to decrease the quantity of web page faults and ensure efficient reminiscence utilization.

## Common Page Replacement Algorithms

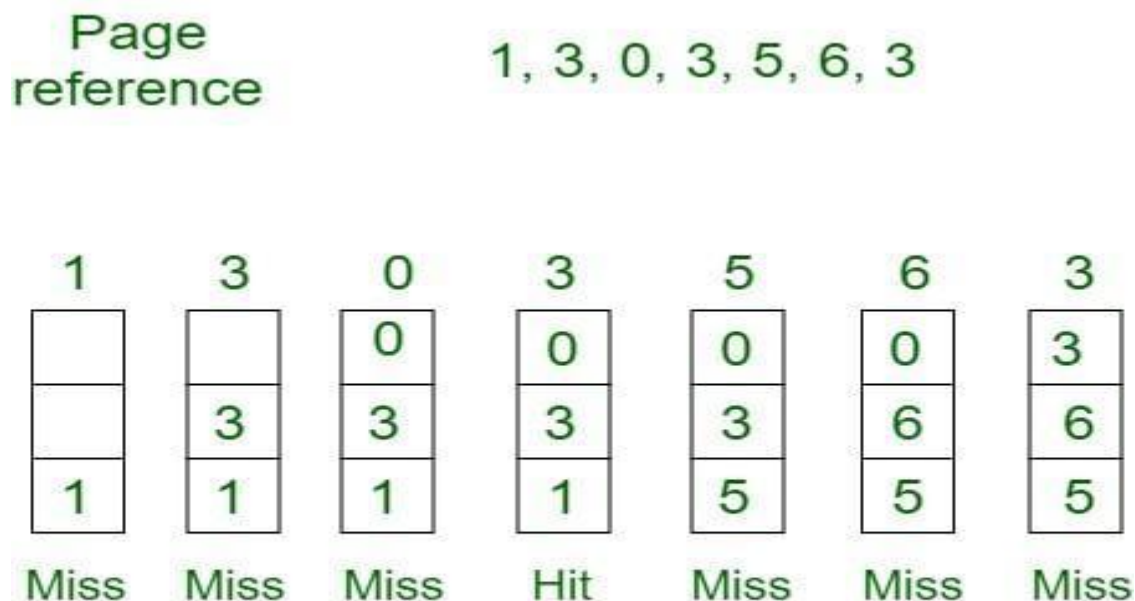
Several page replacement algorithms have been developed over the years, each with its advantages and disadvantages. The most common ones include:

1. First-In-First-Out (FIFO)
2. Optimal Page Replacement (OPT)
3. Least Recently Used (LRU)
4. Clock (Second Chance)
5. Least Frequently Used (LFU)
6. Not Recently Used (NRU)
7. Aging

## First In First Out (FIFO)

This is the simplest page replacement algorithm. In this algorithm, the operating system keeps track of all pages in the memory in a queue, the oldest page is in the front of the queue. When a page needs to be replaced page in the front of the queue is selected for removal.

**Example 1:** Consider page reference string 1, 3, 0, 3, 5, 6, 3 with 3-page frames. Find the number of page faults using FIFO Page Replacement Algorithm.



**Total Page Fault = 6**

- Initially, all slots are empty, so when 1, 3, 0 came they are allocated to the empty slots ---> **3 Page Faults.**
- When 3 comes, it is already in memory so ---> **0 Page Faults.**
- Then 5 comes, it is not available in memory, so it replaces the oldest page slot i.e 1. ---> **1 Page Fault.**
- 6 comes, it is also not available in memory, so it replaces the oldest page slot i.e 3 ---> **1 Page Fault.**
- Finally, when 3 come it is not available, so it replaces 0 1-page **fault.**

Page replacement is a widely used approach in memory management. The idea is simple: the main memory cannot hold all the referred pages in memory. So whenever a new page is referenced, an existing page is replaced by the page that was newly called. The goal of any page replacement mechanism is to minimize the number of page faults.

## Least Recently Used (LRU)

Algorithm is a page replacement technique used for memory management. According to this method, the page which is least recently used is replaced. Therefore, in memory, any page that has been unused for a longer period of time than the others is replaced.

This idea is somewhat based on *locality of reference*, that any page that has been unused for a great amount of time is more likely to remain unused. Therefore, it is better to replace that page.

### Algorithm Example

Let's look at an example:

| Reference string |   |   |   |   |   |   |   |   |   |   |   |
|------------------|---|---|---|---|---|---|---|---|---|---|---|
| 0                | 1 | 2 | 3 | 0 | 1 | 4 | 0 | 1 | 2 | 3 | 4 |
| 0                | 0 | 0 | 3 | 3 | 3 | 4 |   |   | 2 | 2 | 2 |
|                  | 1 | 1 | 1 | 0 | 0 | 0 |   |   | 0 | 3 | 3 |
|                  |   | 2 | 2 | 2 | 1 | 1 |   |   | 1 | 1 | 4 |

### Page frames

Given a reference string: 0, 1, 2, 3, 0, 1, 4, 0, 1, 2, 3, 4 and given that there are a total of 3 memory locations available.

1. The first three pages, 0, 1, 2, will give rise to 3-page faults (look at the image above).
2. Now that the table is filled and the next page, 3, is not present in the table, a page must be replaced. Since 0 is the least recently used page, page 3 will replace 0. This will result in a 1 page fault.
3. This process is repeated for the entire reference string.
4. When a number from the reference string already exists in the table, there is no page fault and no effort is required to replace the page. In that case, the reference page will become the recently used page (like when 0 is referenced when the page frames contain 4, 0 and 1.)
5. Make sure you look at whether or not a given page is recently used from the reference string.

# Definition: LRU (Least Recently Used) Paging

LRU (Least Recently Used) Paging is a page replacement algorithm used in operating systems to manage the contents of the memory. It works by tracking the order in which pages are accessed and replacing the least recently used page when a new page needs to be loaded into memory.

## Understanding LRU (Least Recently Used) Paging

LRU paging is a fundamental concept in memory management, crucial for optimizing the performance of computing systems. This algorithm prioritizes pages based on their usage history, aiming to keep the most recently accessed pages in memory and discarding the ones that haven't been used for the longest time.

### How LRU Paging Works

The LRU algorithm involves maintaining an ordered list of pages based on their recent usage. When a page is accessed, it is moved to the front of the list. If the memory is full and a new page needs to be loaded, the page at the end of the list (the least recently used one) is replaced. This can be implemented using various data structures, such as linked lists, stacks, or more advanced structures like hash maps combined with doubly linked lists for efficient access and updates.

### Benefits of LRU Paging

1. **Efficiency in Access Patterns:** LRU paging adapts well to real-world access patterns, where recently accessed pages are likely to be accessed again soon.
2. **Simple Implementation:** The concept is straightforward, making it relatively easy to implement and understand.
3. **Performance Improvement:** By keeping frequently accessed pages in memory, LRU can significantly reduce page faults, leading to better system performance.

### Use Cases of LRU Paging

LRU paging is widely used in various applications where efficient memory management is critical:

1. **Operating Systems:** To manage the virtual memory and optimize performance.
2. **Database Management Systems:** For caching frequently accessed data to speed up query processing.
3. **Web Caches:** To store web pages and resources, ensuring quick access to frequently visited sites.
4. **Embedded Systems:** Where memory resources are limited and need to be efficiently managed.

## Features of LRU Paging

1. **Recency-Based Replacement:** Prioritizes pages based on their recent access history.
2. **Adaptability:** Can dynamically adapt to changing access patterns.
3. **Scalability:** Suitable for various scales, from small embedded systems to large server environments.

## Implementation of LRU Paging

Implementing LRU paging can be done using several methods:

1. **Linked List:** Maintaining a doubly linked list where each node represents a page. On access, the node is moved to the front, and on replacement, the last node is removed.
2. **Stack:** Using a stack data structure where pages are pushed on access and popped for replacement. This method, however, can be inefficient due to frequent reordering.
3. **Hash Map with Doubly Linked List:** A more efficient approach involves using a hash map to store page references and a doubly linked list to maintain the order of access. This allows  $O(1)$  time complexity for both access and replacement operations.

## Challenges in LRU Paging

Despite its advantages, LRU paging has some challenges:

1. **Overhead:** Maintaining the access order can introduce additional overhead, especially in large systems.
2. **Suboptimal in Some Scenarios:** In certain access patterns, such as those with a high degree of locality, LRU might not perform optimally.

3. **Complexity in Implementation:** More efficient implementations can be complex, requiring sophisticated data structures.

## LRU vs. Other Paging Algorithms

Comparing LRU with other paging algorithms highlights its unique advantages and disadvantages:

1. **FIFO (First In, First Out):** FIFO is simpler but doesn't consider recency, often leading to suboptimal performance.
2. **Optimal Page Replacement:** Theoretically the best but impractical as it requires future knowledge of page references.
3. **LFU (Least Frequently Used):** Focuses on frequency rather than recency, which can be beneficial in different scenarios.

## Optimizations for LRU Paging

Various optimizations can enhance the performance of LRU paging:

1. **Aging:** Introducing a mechanism to age pages gradually, combining recency and frequency.
2. **Segmentation:** Dividing memory into segments and applying LRU within each segment to reduce overhead.
3. **Adaptive Replacement Cache (ARC):** Combining LRU and LFU principles to adapt dynamically to different access patterns.

## Practical Applications and Examples

1. **Web Browsers:** Modern web browsers use LRU caching to store recently visited web pages, improving load times and user experience.
2. **File Systems:** Operating systems use LRU paging to manage file access, ensuring that frequently used files are readily available.
3. **Virtual Machines:** Virtualization platforms implement LRU paging to manage the memory of virtual machines, optimizing performance and resource utilization.

## Optimal Page Replacement Algorithm

- 
-

In operating systems, whenever a new page is referred and not present in memory, page fault occurs, and Operating System replaces one of the existing pages with newly needed page. Different page replacement algorithms suggest different ways to decide which page to replace. The target for all algorithms is to reduce number of page faults. In this algorithm, OS replaces the page that will not be used for the longest period of time in future.

- Optimal page replacement needs to know which pages will be used in the future, which is not possible in real-world scenarios.
- In practical systems, it is impossible to predict exactly which pages will be needed later.
- It is mainly used to analyze and measure the efficiency of practical algorithms, not for actual implementation.

Optimal Page Replacement is one of the Algorithms of Page Replacement. In this algorithm, pages are replaced which would not be used for the longest duration of time in the future.

The idea is simple, for every reference we do following:

1. If referred page is already present, increment hit count.
2. If not present, find if a page that is never referenced in future. If such a page exists, replace this page with new page. If no such page exists, find a page that is referenced farthest in future. Replace this page with new page.

Let's consider some examples:

### **Example 1:**

Consider the page references 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 3 with 4-page frame. Find number of page fault using Optimal Page Replacement Algorithm.

| 7    | 0    | 1    | 2    | 0   | 3    | 0   | 4    | 2   | 3   | 0   | 3   | 2   | 3   |
|------|------|------|------|-----|------|-----|------|-----|-----|-----|-----|-----|-----|
|      |      |      | 2    | 2   | 2    | 2   | 2    | 2   | 2   | 2   | 2   |     |     |
|      |      | 1    | 1    | 1   | 1    | 1   | 4    | 4   | 4   | 4   | 4   | 4   | 4   |
|      | 0    | 0    | 0    | 0   | 0    | 0   | 0    | 0   | 0   | 0   | 0   | 0   | 0   |
| 7    | 7    | 7    | 7    | 7   | 3    | 3   | 3    | 3   | 3   | 3   | 3   | 3   | 3   |
| Miss | Miss | Miss | Miss | Hit | Miss | Hit | Miss | Hit | Hit | Hit | Hit | Hit | Hit |

### Example 2:

Input: Number of frames = 3, Reference String = {7, 0, 1, 2, 0, 3, 0, 4, 2, 3}

#### Page-by-page analysis:

- **7, 0, 1, 2:** First four pages fill the frames → Misses = 4
- **0:** Already in the frame → Hit
- **3:** Replaces 7 (not used for the longest) → Miss
- **0:** Already in the frame → Hit
- **4:** Replaces 1 → Miss
- **2, 3:** All already in frames → Hits = 2

| 7 | 0 | 1 | 2 | 0 | 3 | 0 | 4 | 2 | 3 |
|---|---|---|---|---|---|---|---|---|---|
|   |   | 1 | 1 | 1 | 3 | 3 | 3 | 3 | 3 |
|   | 0 | 0 | 0 | 0 | 0 | 0 | 4 | 4 | 4 |
| 7 | 7 | 7 | 2 | 2 | 2 | 2 | 2 | 2 | 2 |
| M | M | M | M | H | M | H | M | H | H |

#### Final Tally:

- **Hits = 4**
- **Misses = 6**

### Example 3:

Input: Number of frames = 3, Reference String = {6, 7, 8, 9, 6, 7, 1, 6, 7, 8, 9, 1, 7, 9, 6}

#### Page-by-page analysis:

- **6, 7, 8, 9:** First four pages fill the frames → Misses = 4
- **6:** Already in the frame → Hit
- **7:** Already in the frame → Hit
- **1:** replaces 9 → Miss
- **6, 7:** Already in the frame → Hit
- **8, 9:** 8 replaces 6 and 9 replaces 8 → Misses = 2
- **1, 7, 9:** already in frames → Hits = 3
- **6:** 6 replaces 9 → Miss



| Pages >> | 6    | 7    | 8    | 9    | 6   | 7   | 1    | 6   | 7   | 8    | 9    | 1   | 7   | 9   | 6    |
|----------|------|------|------|------|-----|-----|------|-----|-----|------|------|-----|-----|-----|------|
| Frame 3  |      |      | 8    | 9    | 9   | 9   | 1    | 1   | 1   | 1    | 1    | 1   | 1   | 1   | 1    |
| Frame 2  |      | 7    | 7    | 7    | 7   | 7   | 7    | 7   | 7   | 7    | 7    | 7   | 7   | 7   | 7    |
| Frame 1  | 6    | 6    | 6    | 6    | 6   | 6   | 6    | 6   | 6   | 8    | 9    | 9   | 9   | 9   | 6    |
|          | Miss | Miss | Miss | Miss | Hit | Hit | Miss | Hit | Hit | Miss | Miss | Hit | Hit | Hit | Miss |

### Final Tally:

- **Hits = 6**
- **Misses = 8**

## Fragmentation in Operating System

- 
- 
- 

Fragmentation in computing is the process in which a file, such as a data file or an executable program, is divided into smaller fragments that are stored in different parts of a storage medium, such as a hard disk or RAM. When a file is fragmented, its fragments are stored in non-contiguous blocks, meaning they are not placed next to each other. This can lead to inefficient use of memory or storage and slower access to files. Fragmentation occurs both in memory (RAM) and storage, and it is a common challenge in operating systems that affects overall system performance.

- Fragmentation occurs when processes repeatedly load and unload from memory, leaving small blocks of memory unused.
- Programs create free spaces or "holes" in memory when they are loaded and removed, which cannot always be used by new processes.
- The memory allocation scheme of the operating system determines how and when fragmentation occurs.
- Fragmented memory or storage becomes inefficient, as the scattered blocks cannot be easily assigned to incoming processes.
- Accessing fragmented files can slow down the system because the OS has to locate and assemble the scattered fragments.
- Defragmentation is the process of reorganizing memory or storage to store files in contiguous blocks, improving efficiency and access speed.

### Cause of Fragmentation

- Fragmentation occurs when a file is too large to fit into a single contiguous block of free space on the storage medium.

- It can also happen when the available free blocks on the storage medium are too small to hold the entire file.
- As a result, the file is split into fragments that are stored in different locations.
- The system must search for and retrieve these scattered fragments to access the file, which can slow down file reading and processing.

## **Effect of Fragmentation**

- Fragmentation reduces overall system performance because accessing scattered fragments takes more time.
- It makes files harder to read or access efficiently.
- Regular defragmentation helps reorganize the scattered data blocks so that files are stored in contiguous blocks.
- Defragmenting improves access speed, memory or storage efficiency, and overall system performance.

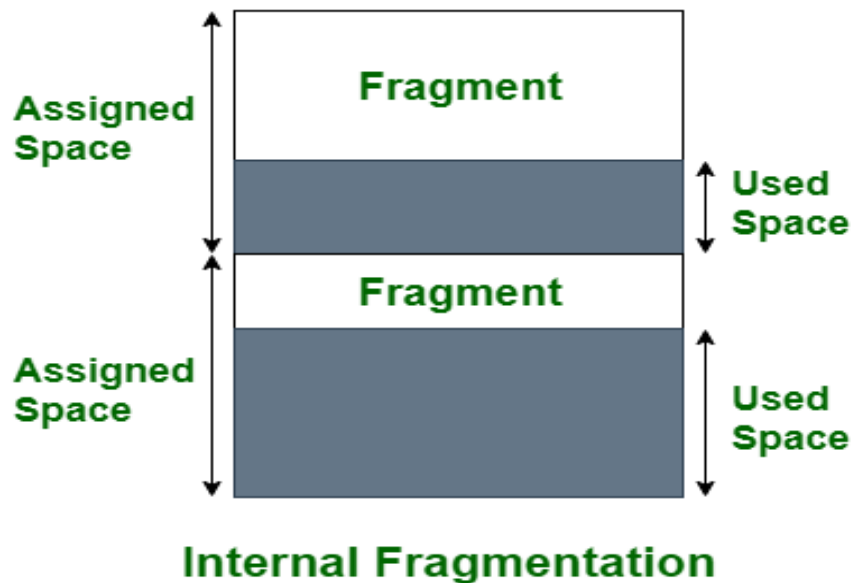
## **Types of Fragmentation**

In an operating system, fragmentation mainly occurs in two different forms based on how memory or storage space is utilized. These types explain where and how unused space is created during memory or storage allocation. The two main types of fragmentation are:

- Internal Fragmentation
- External Fragmentation

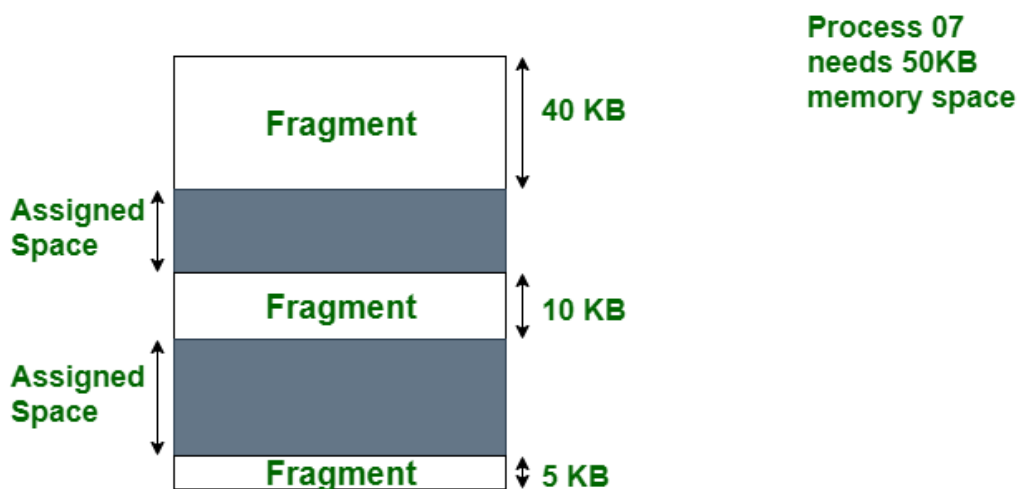
### **1. Internal Fragmentation**

- Internal fragmentation occurs when allocated memory contains unused space within a block.
- It happens when the size of the allocated memory block is larger than the actual memory required by a process.
- For example, if a system allocates a 64 KB memory block to a process that requires only 40 KB, the remaining 24 KB remains unused, resulting in internal fragmentation.
- This type of fragmentation commonly occurs in systems that use fixed-size memory allocation techniques.
- The unused space inside the allocated block cannot be utilized by other processes, leading to inefficient memory usage.



## External Fragmentation

- External fragmentation occurs when free memory or storage space is divided into many small, non-contiguous blocks.
- It is caused by frequent allocation and deallocation of processes or files over time.
- Even if the total free space is sufficient, the system may be unable to allocate memory because no single contiguous block is large enough.
- As a result, files or processes must be stored in multiple smaller blocks, increasing access time.
- External fragmentation leads to performance degradation and inefficient use of storage or memory resources.



Fragmentation can also occur at various levels within a system. File fragmentation, for example, can occur at the file system level, in which a

file is divided into multiple non-contiguous blocks and stored on a storage medium. Memory fragmentation can occur at the [memory management](#) level, where the system allocates and deallocated memory blocks dynamically. Network fragmentation occurs when a packet of data is divided into smaller fragments for transmission over a network.

## **Impact of Fragmentation on Operating System Performance**

- Fragmentation significantly affects the overall performance and stability of an operating system.
- It slows down disk read and write operations because the disk head must move to multiple locations to access scattered file fragments.
- Increased disk head movement raises access time, which reduces system speed and causes application slowdowns and lag.
- Fragmentation leads to inefficient disk space utilization, as fragmented files may occupy more space than necessary.
- Wasted disk space can result in storage shortages, making the system unstable and more prone to errors or crashes.
- In severe cases, excessive fragmentation can cause the system to run out of disk space, increasing the risk of data loss.
- A highly fragmented disk increases the likelihood of system crashes and operational errors.
- Fragmentation can negatively impact battery life in laptops and mobile devices because the disk has to work harder to access scattered data.
- Regular defragmentation reorganizes fragmented files into contiguous blocks, improving disk access speed and reducing latency.
- Maintaining low fragmentation ensures efficient resource usage, stable system operation, and a smooth user experience.

## **Advantages**

- Fragmentation allows better utilization of available storage space on a hard disk or other storage media. Files can be stored in free blocks even if those blocks are not contiguous, preventing small unused spaces from being wasted.
- It enables the system to efficiently use scattered free space, which is especially helpful when the storage medium contains many small free blocks.
- Fragmentation makes it possible to store large files even when a single contiguous block of free space is not available. The file can be divided and stored across multiple smaller blocks.
- It provides flexibility in memory and storage allocation, allowing the operating system to continue functioning without requiring large continuous memory regions.

- Although fragmentation has these advantages, it is generally recommended to minimize it because excessive fragmentation can reduce system performance and make file access and management more complex.

## Disadvantages

- Fragmentation can significantly degrade system performance, especially during file reading or access operations. The system must locate and retrieve multiple fragments from different locations, which takes more time than accessing a single contiguous file.
- It makes file management and organization more complex, as fragmented files are stored in multiple non-contiguous blocks, making them harder to track and access efficiently.
- Fragmentation can reduce the lifespan of storage devices such as hard disks and solid-state drives. Frequent access to scattered data increases read/write operations, leading to additional wear and tear on the storage medium.
- It consumes extra storage space because the system needs to maintain additional metadata to track the locations of each fragment. This reduces the amount of available space for storing other files.

## First-Fit Allocation in Operating Systems

- 
- 
- 

In an operating system (OS), memory management is a critical function that ensures efficient allocation and utilization of memory resources. When processes are initiated, they require execution time and storage space, which is allocated in memory blocks. The OS uses various memory allocation algorithms to manage the assignment of memory to these incoming processes.

The following are the most used algorithms:

- 1. First Fit**
- 2. Best Fit**
- 3. Worst Fit**
- 4. Next Fit**

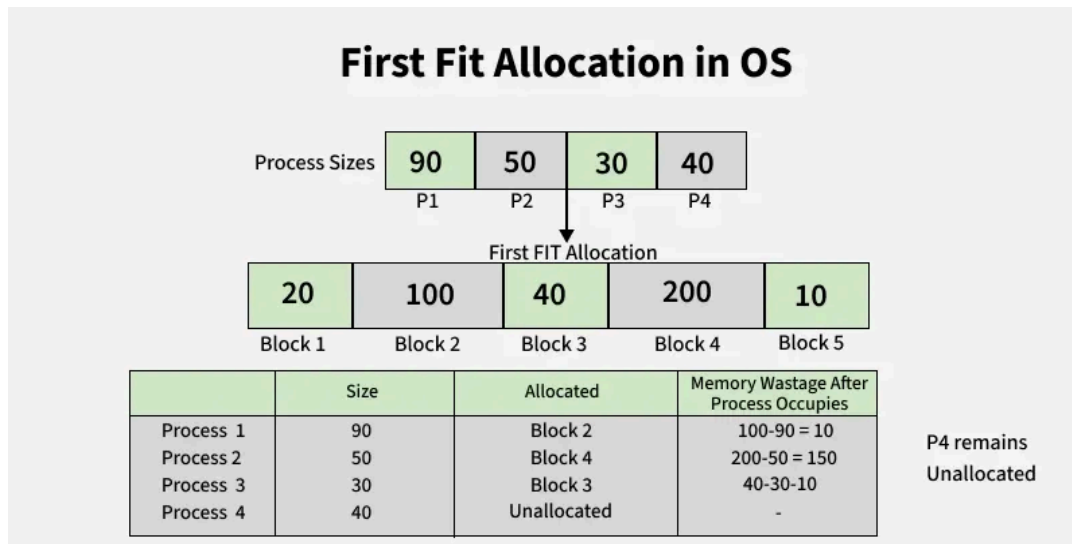
These are [Contiguous memory allocation](#) techniques.

### First-Fit Memory Allocation

The **First Fit** method works by searching through memory blocks from the beginning, looking for the first block that can fit the process.

- For any process  $P_n$ , the operating system looks through the available memory blocks.
- It allocates the first block that is free and large enough to hold the process.

In simple terms, First Fit simply finds the first available block that can fit the process and assigns it there. It's a straightforward and fast way to allocate memory.



## Advantages of First Fit Algorithm

The First Fit algorithm in operating systems offers several benefits:

1. It is straightforward to implement and easy to understand, making it ideal for systems with limited computational power.
2. Memory can be allocated quickly when a suitable free block is found at the start.
3. When processes have similar memory sizes, First Fit can help minimize fragmentation by utilizing the first available block that fits the process.

## Disadvantages of First Fit Algorithm

Despite its advantages, the First Fit algorithm has a few downsides:

1. Over time, First Fit can lead to both external fragmentation, where small free memory blocks are scattered, and internal fragmentation, where allocated memory exceeds the process's requirement, wasting space.
2. It may not always allocate memory in the most efficient manner, leading to suboptimal use of available memory.
3. For large processes, First Fit can be less efficient, as it may need to search through numerous smaller blocks before finding an appropriate one, which can slow down memory allocation.

## Partition Allocation Methods in Memory Management

- 
- 
- 

Memory management techniques are methods used by an operating system to allocate, control, and release main memory during program execution. They ensure that multiple processes can use memory efficiently without interfering with each other. Proper memory management improves system performance and prevents memory wastage.

- **Single contiguous allocation:** Simplest allocation method used by MS-DOS. All memory (except some reserved for OS) is available to a process.
- **Partitioned allocation:** Memory is divided into different blocks or partitions. Each process is allocated according to the requirement.
- **Paged memory management:** Memory is divided into fixed-sized units called page frames, used in a virtual memory environment.
- **Segmented memory management:** Memory is divided into different segments (a segment is a logical grouping of the process' data or code). In this management, allocated memory doesn't have to be contiguous.

Most of the operating systems (for example Windows and Linux) use Segmentation with Paging. A process is divided into segments and individual segments have pages.

*In **Partition Allocation**, when there is more than one partition freely available to accommodate a process's request, a partition must be selected. To choose a particular partition, a partition allocation method is needed. A partition allocation method is considered better if it avoids internal fragmentation.*

## There are different Placement Algorithm:

1. First Fit
2. Best Fit
3. Worst Fit
4. Next Fit

**1. First Fit:** In the first fit, the partition is allocated which is the first sufficient block from the top of Main Memory. It scans memory from the beginning and chooses the first available block that is large enough. Thus it allocates the first hole that is large enough.

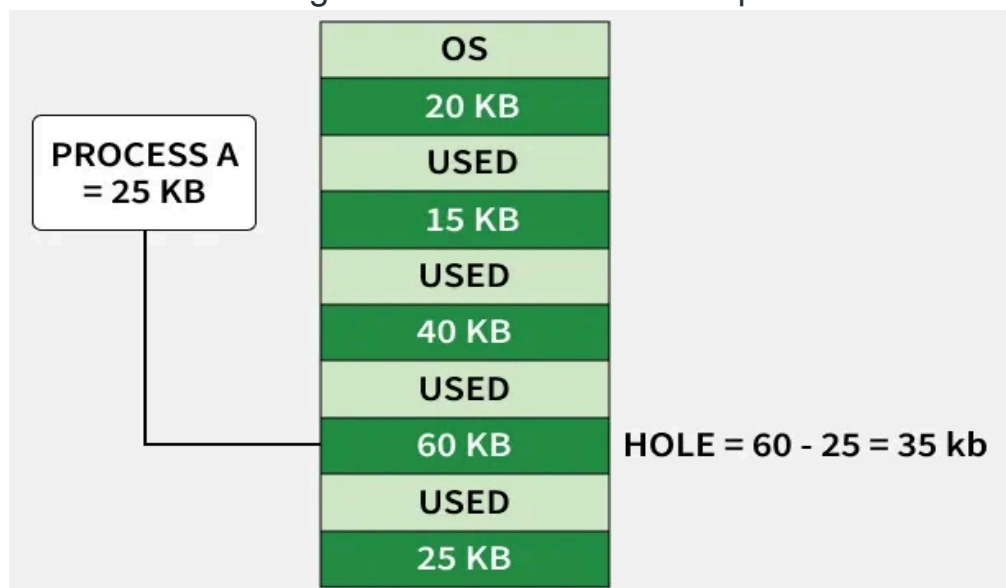


**2.Best Fit** Allocate the process to the partition which is the first smallest sufficient partition among the free available partition. It searches the entire list of holes to find the smallest hole whose size is greater than or equal to the size of the process.





**3. Worst Fit** Allocate the process to the partition which is the largest sufficient among the freely available partitions available in the main memory. It is opposite to the best-fit algorithm. It searches the entire list of holes to find the largest hole and allocate it to process.



**4. Next Fit:** Next fit is similar to the first fit but it will search for the first sufficient partition from the last allocation point.

### Comparison of Partition Allocation Methods:

| Sl.No. | Partition Allocation Method | Advantages                                                    | Disadvantages                                      |
|--------|-----------------------------|---------------------------------------------------------------|----------------------------------------------------|
| 1.     | Fixed Partition             | Simple, easy to use, no complex algorithms needed             | Memory waste, inefficient use of memory resources  |
| 2.     | Dynamic Partition           | Flexible, more efficient, partitions allocated as required    | Requires complex algorithms for memory allocation  |
| 3.     | Best-fit Allocation         | Minimizes memory waste, allocates smallest suitable partition | More computational overhead to find smallest split |
| 4.     | Worst-fit Allocation        | Ensures larger processes have sufficient memory               | May result in substantial memory waste             |
| 5.     | First-fit Allocation        | Quick, efficient, less computational work                     | Risk of memory fragmentation                       |

### Exercise:

Consider the memory requests from processes in the following order:

**300K, 25K, 125K, 50K.**

The available memory blocks are:

**150K, 150K, 350K.**

Which of the following partition allocation schemes can satisfy the above requests?

- A) Best fit but not first fit
- B) First fit but not best fit
- C) Both first fit & best fit
- D) Neither first fit nor best fit

### Solution:

**Best Fit:**

1. 300K → allocated from 350K block, leaving 50K.
2. 25K → allocated from the remaining 50K block, leaving 25K.
3. 125K → allocated from 150K block, leaving 25K.
4. 50K → cannot be allocated (remaining free spaces are 25K + 25K, which is insufficient).

**Best fit cannot satisfy all requests.**

**First Fit:**

1. 300K → allocated from 350K block, leaving 50K.
2. 25K → allocated from 150K block, leaving 125K.
3. 125K → allocated from the remaining 125K block.
4. 50K → allocated from the remaining 50K block.

First fit can satisfy all requests.

Answer:

B) First fit but not best fit